

Como trabalhar com listas em Python

Veja nesse artigo como criar e manipular dados na forma de listas na linguagem de programação Python.

Python List

Durante o desenvolvimento de software, independentemente de plataforma e linguagem, é comum a necessidade de lidar com listas. Por exemplo, elas podem ser empregadas para armazenar contatos telefônicos ou tarefas.

Uma lista é uma estrutura de dados composta por itens organizados de forma linear, na qual cada um pode ser acessado a partir de um índice, que representa sua posição na coleção (iniciando em zero).

Neste artigo vamos aprender a **trabalhar com listas** em [Python](#), linguagem na qual essa estrutura de dados pode armazenar, inclusive simultaneamente, objetos de diferentes tipos, como strings, números e até mesmo outras listas.

Criando listas

Em **Python**, uma **lista é representada como uma sequência de objetos** separados por vírgula e dentro de colchetes [], assim, uma lista vazia, por exemplo, pode ser representada por colchetes sem nenhum conteúdo. A **Listagem 1** mostra algumas possibilidades de criação desse tipo de objeto.

```
1 01 >>> lista = []
2 02 >>> lista
3 03 []
4 04 >>> lista = ['O carro', 'peixe', 123, 111]
5 05 >>> lista
6 06 ['O carro', 'peixe', 123, 111]
7 07 >>> nova_lista = ['pedra', lista]
8 08 >>> nova_lista
9 09 ['pedra', ['O carro', 'peixe', 123, 111]]
```

Listagem 1. Criando e inicializando listas

Linhas 1 a 3: Declaração de uma lista sem nenhum elemento e sua impressão, que apresenta apenas os colchetes, indicando que a lista está vazia;

Linhas 4 a 6: Atribuição de duas strings e dois inteiros à variável lista e sua posterior impressão;

Linhas 7 a 9: Criação e impressão da variável nova_lista, que é inicializada com a string 'pedra', e uma outra lista.

As possibilidades de declaração e atribuição de valores a uma lista são várias, e a opção por uma ou outra dependerá do contexto e aplicação.

Operadores de lista

A [linguagem Python](#) dispõe de vários métodos e operadores para auxiliar na manipulação de listas. O primeiro e mais básico é o operador de acesso aos seus itens a partir dos índices. Para compreendê-lo, é importante entender como os dados são armazenados nessa estrutura, o que é ilustrado na **Figura 1**.

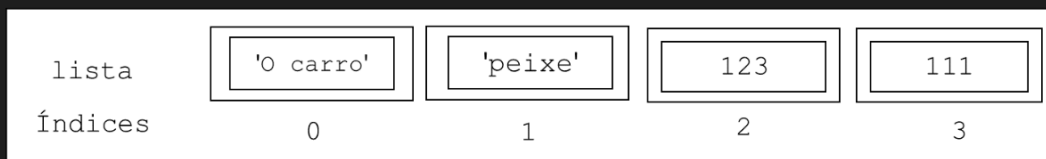


Figura 1. Representação de uma lista e os índices dos elementos

A lista representada nessa figura é composta por quatro elementos, cujos índices variam de zero a três. Por exemplo, o primeiro objeto, no índice 0, é a string 'O carro'. Tendo em mente essa organização interna, podemos acessar cada um dos elementos utilizando a sintaxe `lista[índice]`, como mostra a **Listagem 2**.

```
1  01 >>> lista[0]
2  02 'O carro'
3  03 >>> lista[2]
4  04 123
5  05 >>> nova_lista[0]
6  06 'pedra'
7  07 >>> nova_lista[1]
8  08 ['O carro azul', 'peixe', 123, 111]
9  09 >>> nova_lista[1][2]
10 10 123
```

Listagem 2. Acessando os itens da lista

Linhas 1 a 8: Acessamos os elementos contidos em determinados índices das variáveis `lista` e `nova_lista`, que são strings (linhas 2 e 6), valores numéricos (linha 4) e uma lista interna (linha 8);

Linhas 9 e 10: Acessamos um elemento da lista interna contida na posição 1 da variável `nova_lista`. Perceba que como esse índice aponta para uma outra lista, é possível acessar seus elementos utilizando a mesma sintaxe.

Comprimento de uma lista

O comprimento de uma lista, ou o número de itens que a compõem, pode ser obtido a partir da função `len()`, como mostra o código abaixo, em que é impresso o valor 2, indicando que a variável `nova_lista` contém 2 elementos.

```
1 | 01 >>> len(nova_lista)
2 | 02 2
```

Concatenação e multiplicação

Também é possível concatenar listas por meio do operador de adição `+` e multiplicá-las por um inteiro, o que gerará várias cópias dos seus itens. O código a seguir traz alguns exemplos dessas operações:

```
1 | 01 >>> lista+nova_lista
2 | 02 ['O carro azul', 'peixe', 123, 111, 'pedra', ['O carro azul', 'peixe'
3 | 03 >>> lista*3
4 | 04 ['O carro azul', 'peixe', 123, 111, 'O carro azul',
5 | 'peixe', 123, 111, 'O carro azul', 'peixe', 123, 111]
```

Linhas 1 e 2: Concatenação de duas listas (`lista+nova_lista`), seguida da impressão do resultado, que contém os itens das duas variáveis;

Linhas 3 e 4: Multiplicação da variável `lista` por 3, e impressão do resultado, com os itens repetidos três vezes.

Verificando a existência de itens em uma lista

Em alguns casos é preciso verificar se um determinado valor está contido em uma lista. Para isso, em Python utilizamos o operador `in`, que indicará `True` se objeto pertencer ao conjunto, e `False` caso contrário. No código a seguir temos exemplos de uso desse operador:

```
1 01 >>> 'peixe' in lista
2 02 True
3 03 >>> 'gato' in lista
4 04 False
```

Valores mínimos, máximos e soma

O Python oferece ainda as funções `min()`, `max()` e `sum()`, através das quais é possível encontrar, respectivamente, o menor valor, o maior valor ou ainda realizar a soma de todos os elementos da lista. Na **Listagem 3** podemos ver como utilizá-las.

```
1 01 >>> numeros = [14.55, 67, 89.88, 10, 21.5]
2 02 >>> min(numeros)
3 03 10
4 04 >>> max(numeros)
5 05 89.88
6 06 >>> sum(numeros)
7 07 202.93
```

Listagem 3. Funções sobre as listas

Métodos das listas

Nas seções anteriores, vimos os operadores e funções que recebem uma lista como argumento, retornam um resultado, mas não efetuam alterações na sua estrutura. A partir de agora veremos métodos pertencentes ao tipo lista, e que nos permitem incluir ou remover elementos, bem como classificar as coleções.

O primeiro método a ser analisado é o `append()`, que tem por objetivo adicionar um novo elemento no final da lista, conforme mostra o código abaixo:

```
1 01 >>> livros = ['Java', 'SqlServer', 'Delphi', 'Python']
2 02 >>> livros.append('Android')
3 03 >>> livros
4 04 ['Java', 'SqlServer', 'Delphi', 'Python', 'Android']
```

Após criar, na primeira linha, uma lista com um conjunto de valores, adicionamos a ela um novo item: “Android”. Como podemos ver na linha 4, o elemento é adicionado após o último item. Caso desejemos fazer essa inserção em uma posição específica, podemos utilizar o método `insert()` que, além do elemento a ser inserido, recebe também o índice que ele deve assumir, como mostra o código abaixo:

```
1 01 >>> livros = ['Java', 'SqlServer', 'Delphi', 'Python', 'Android']
2 02 >>> livros.insert(0, 'Oracle')
3 03 >>> livros
4 04 ['Oracle', 'Java', 'SqlServer', 'Delphi', 'Python', 'Android']
```

O próximo método importante é o `pop()`, que remove o último item da lista e o retorna como resultado da operação. Caso seja necessário remover um índice específico, basta informá-lo como argumento. Na **Listagem 4** temos alguns exemplos de uso.

```

1 01 >>> livros
2 02 ['Java', 'SqlServer', 'Delphi', 'Python', 'Android']
3 03 >>> livros.pop()
4 04 'Android'
5 05 >>> livros.pop(1)
6 06 'SqlServer'
7 07 >>> livros
8 08 ['Java', 'Python', 'Delphi']

```

Listagem 4. Removendo valores com o método pop()

Na linha 3 removemos o último elemento, ou seja, a string “Android”, já na linha 5 removemos o item na posição 1 (o segundo item). Nos dois casos, a remoção foi feita com base na posição do objeto, porém, há situações em que esse índice é desconhecido e desejamos remover o item a partir do seu valor. Para isso temos o método `remove(item)` que, conforme mostra a **Listagem 5**, remove a primeira ocorrência do item passado como parâmetro.

```

1 01 >>> livros
2 02 ['Oracle', 'Java', 'SqlServer', 'Delphi', 'Python', 'Android', 'Oracle']
3 03 >>> livros.remove('Oracle')
4 04 >>> livros
5 05 ['Java', 'SqlServer', 'Delphi', 'Python', 'Android', 'Oracle']
6 06 >>> livros.remove('Oracle')
7 07 >>> livros
8 08 ['Java', 'SqlServer', 'Delphi', 'Python', 'Android']
9 09 >>> livros.remove('Oracle')
10 10 Traceback (most recent call last):
11 11   File "<pyshell#34>", line 1, in <module>
12 12     livros.remove('Oracle')
13 13 ValueError: list.remove(x): x not in list

```

Listagem 5. Removendo a primeira ocorrência de um item

Listagem 5. Removendo a primeira ocorrência de um item

Linhas 1 e 2: Verificamos o conteúdo da variável `livros`, que contém duas ocorrências da string “Oracle”;

Linha 3: Removemos a primeira ocorrência de “Oracle”, ou seja, aquela do índice 0;

Linhas 4 e 5: Imprimimos o valor da variável `livros`, que agora possui um elemento a menos e mantém o item “Oracle” na última posição;

Linha 6: Removemos novamente a primeira ocorrência da string “Oracle”, que agora é o último item;

Linhas 7 e 8: Imprimimos novamente a lista, que agora não mais contém “Oracle” entre seus itens;

Linha 9: Tentamos excluir novamente o item “Oracle”, que não mais existe na coleção;

Linhas 10 a 13: Um erro ocorre, informando que o item passado por parâmetro não existe na lista `livros`.

Outros métodos muito importantes ao trabalhar com listas são o `sort()`, que ordena a lista, e o `reverse()`, que inverte as posições dos itens. No caso da ordenação, ela será feita em ordem crescente para números e em ordem lexicográfica para strings, ou seja, na forma como são ordenadas no dicionário. A **Listagem 6** traz alguns exemplos de uso desses métodos.

```
1 01 >>> livros
2 02 ['Java', 'SqlServer', 'Delphi', 'Python', 'Android']
3 03 >>> livros.reverse()
4 04 >>> livros
5 05 ['Android', 'Python', 'Delphi', 'SqlServer', 'Java']
6 06 >>> livros.sort()
7 07 >>> livros
8 08 ['Android', 'Delphi', 'Java', 'Python', 'SqlServer']
```

Listagem 6. Ordenando e invertendo a lista

Por fim, temos ainda o método `count()`, que retorna o número de ocorrências de determinado objeto, passado como parâmetro, em uma lista. Na **Listagem 7** podemos ver seu funcionamento.

```
1 01 >>> livros = ['Oracle', 'Java', 'SqlServer', 'Delphi', 'Python', 'And
2 02 >>> livros.count('Python')
3 03 1
4 04 >>> livros.count('Oracle')
5 05 2
```

Listagem 7. Contando objetos

```
1 01 >>> livros = ['Oracle', 'Java', 'SqlServer', 'Delphi', 'Python', 'And
2 02 >>> livros.count('Python')
3 03 1
4 04 >>> livros.count('Oracle')
5 05 2
```

Listagem 7. Contando objetos

Nesse exemplo, identificamos que há apenas uma ocorrência da string `Python`, ao passo que, na linha 5, vemos que o termo “Oracle” aparece duas vezes na lista.

O uso de listas é bastante comum em diversos tipos de aplicação, portanto é fundamental saber lidar com essa estrutura, utilizando para isso os diversos recursos oferecidos pela linguagem.